Progress Report

Discrete State-Space Models and Expectation-Maximization for Cell-State Trajectories

Arnab Kumar Ghosh · Supervisor: Prashant Singh · MSc Computer Science, Uppsala University · Weeks 5–6 · November 10, 2026

Overview

The main open question of the last report was how to build the observation sequence. In the meeting on 6 October the supervisor and I worked through the population-level model on the whiteboard. In weeks 5–6 I wrote this model down as a likelihood, with its E-step and M-step (Sections 1–4), the starting values (Section 5), what the data can and cannot determine (Section 6), and a plan to test it on a toy example (Section 7). Figure 1 summarises the model and the fitting procedure. Progress against the plan from the last report:

1. Bilmes (1998) and Gaussian emissions in the toy code: **started** implementing the toy example of the population model (Section 7).
2. Write down the population-level likelihood, E-step and M-step with the supervisor: **done** (Sections 1–4).
3. Open `adata_final.h5ad`, check the normalisation, compute state proportions per time point: **started** looking at the data; I contacted the previous student for an overview of it.
4. Read Amoedo-Leite et al. (2024): **started** reading.
5. Validation set-up: **written down** (Section 6), not yet implemented.

1 The population-level model

What I learned

Sequencing destroys the cell, so each cell is measured once and no cell can be followed over time. A standard HMM needs one system observed again and again, so no sequence of individual cells can be built. What the data do support is the proportion of cells in each state on each day. The model therefore follows the population, not single cells. Let $p_t = (p_t[\mathrm{MA}], p_t[\mathrm{T}], p_t[\mathrm{MU}])$ be the state proportions in week t. They are linked by one transition matrix:

$$p_{t+1} = p_t \cdot A, \qquad p_0 = \pi, \qquad \text{so} \qquad p_t = \pi \cdot A^t.$$

Written out: $p_1 = p_0 A = \pi A$, $p_2 = p_1 A = \pi A^2$, $p_3 = p_2 A = \pi A^3$. Each week adds one factor of A, so the power of A equals the number of weeks since day 0. One step is one week: $t = 0, 1, 2, 3$ for day 0, 7, 14 and 21. The same A is used for every week. p_t is a row vector.

$A[i, j]$ is now the fraction of the state-i population that is in state j one week later, not the path of one cell. The off-diagonal entries give the direction and the speed of reprogramming. π is learned, not fixed. Most cells should start as macrophages, so π should come out close to $(1, 0, 0)$, but not exactly.

In the project

This replaces the sequence $O = o_1\, o_2\, o_3$ (day 7, 14, 21) of the last report. Day 0 is the healthy tissue, before injury, so the healthy cells enter the model as the first time point. Section 6 shows why this time point is needed.

2 Gaussian emissions

What I learned

Each state j has one Gaussian over the expression vector $\mathbf{x}_c$ of a cell, shared across all days. Here z_c is the hidden state of cell c, and d is the number of genes:

$$\mathbf{x}_c \mid z_c = j \;\sim\; \mathcal{N}(\mu_j, U_j), \qquad U_j = \mathrm{diag}(\sigma^2_{j,1}, \ldots, \sigma^2_{j,d}).$$

$\mathcal{N}(\mu_j, U_j)$ is the multivariate normal (Gaussian) distribution with mean vector μ_j and covariance matrix U_j:

- Mean vector: $\mu_j = (\mu_{j,1}, \ldots, \mu_{j,d})$ holds the typical expression of each gene in state j.
- Covariance matrix: U_j is a $d \times d$ matrix. Its diagonal entry $\sigma^2_{j,g}$ is the variance of gene g in state j. Its off-diagonal entry (g, h) is the covariance of genes g and h, i.e. how much they rise and fall together within the state.
- Diagonal covariance: I set all off-diagonal entries to zero. Within a state, the genes are then treated as independent, and only the d variances are learned.

The formula has no t in it: the same three Gaussians are used on every day, and only the mixture weights change. With a diagonal U_j the density is a product of one-gene densities, so its log is a sum over the genes:

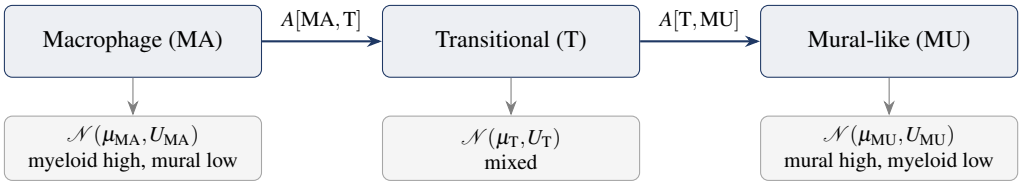
$$\log \mathcal{N}(\mathbf{x}_c; \mu_j, U_j) = \sum_{g=1}^{d} \Big[-\tfrac{1}{2} \log(2\pi\sigma^2_{j,g}) - \frac{(x_{c,g} - \mu_{j,g})^2}{2\sigma^2_{j,g}} \Big].$$

The formula comes from three steps:

- One gene: the density of gene g in state j is the 1D Gaussian $\frac{1}{\sqrt{2\pi\sigma^2_{j,g}}} \exp\big(-\frac{(x_{c,g} - \mu_{j,g})^2}{2\sigma^2_{j,g}} \big)$.
- Log: the fraction becomes the size term $-\frac{1}{2}\log(2\pi\sigma^2_{j,g})$, and the exponential leaves the distance term $-\frac{(x_{c,g} - \mu_{j,g})^2}{2\sigma^2_{j,g}}$.
- All genes: with a diagonal U_j the genes are independent within a state, so the density of the cell is a product over genes, and its log is a sum over genes.

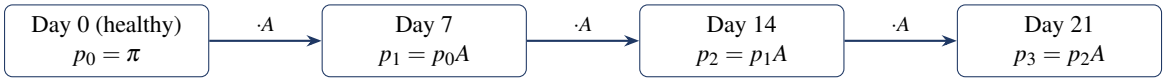
The distance term compares the value of gene g in the cell, $x_{c,g}$, with the typical value of that gene in state j, $\mu_{j,g}$. The difference is squared and divided by the variance, so it is measured relative to how much the gene normally varies in that state. If the value

(a) Hidden states and Gaussian emissions



Hidden: the state z_c of each cell. Observed: its 50-gene expression vector $\mathbf{x}_c$. Arrows show the main path; A holds all nine transitions.

(b) Population proportions over the four days



Each day: new cells, each measured once. A cell from day t is drawn from the mixture $\sum_j p_t[j]\,\mathcal{N}(\mu_j, U_j)$.

(c) Fitting with EM

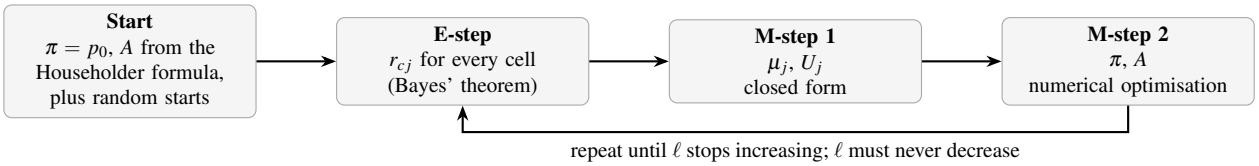


Figure 1: (a) Three hidden states, each with one Gaussian over the expression vector, shared across all days. (b) No cell is followed over time; the state proportions of the population move from week to week by one transition matrix A. (c) EM fits π, A, μ and U together.

is close to the mean, the term is close to zero. The further the value is from the mean, the more negative the term becomes, and the less the cell fits state j. The size term depends only on the spread: a wider Gaussian has a lower peak. The sum adds the evidence of all d genes.

Two reasons for this form. Diagonal: a full 50×50 covariance has $50 \times 50 = 2500$ entries, but it is symmetric: the covariance of genes g and h equals that of h and g, so every off-diagonal number appears twice. The free numbers are the 50 variances on the diagonal plus $(2500-50)/2 = 1225$ covariances, in total $d(d+1)/2 = 1275$ per state. A diagonal covariance sets all covariances to zero and keeps only the 50 variances. Log form: multiplying 50 small densities underflows to zero, adding 50 logs does not.

In the project

$d = 50$: the top 25 marker genes of cluster 2 and of cluster 8. I will use log-normalised values.

3 Likelihood and E-step

What I learned

A cell measured in week t comes from a mixture of the three Gaussians, with the weights p_t of that week. Given the parameters $\lambda = (\pi, A, \mu, U)$, the cells are independent, so the log-likelihood is a sum over cells:

$$\ell(\lambda) = \sum_c \log \sum_{j=1}^{3} p_{t_c}[j]\,\mathcal{N}(\mathbf{x}_c; \mu_j, U_j).$$

The weights of the four days are not free: they all come from one π and one A. This is the time coupling. Without it, the model would be four unrelated mixture models. The inner sum is computed with log-sum-exp, a way to add the terms in log space without underflow.

The E-step is Bayes' theorem for each cell. The prior is the proportion of the cell's week, and the likelihood is the Gaussian density:

$$r_{cj} = \frac{p_{t_c}[j]\,\mathcal{N}(\mathbf{x}_c; \mu_j, U_j)}{\sum_k p_{t_c}[k]\,\mathcal{N}(\mathbf{x}_c; \mu_k, U_k)}.$$

r_{cj} plays the role of $\gamma_t(i)$, but no forward–backward pass is needed. The day matters: the same expression vector can get different r on different days, because the prior p_{t_c} differs from week to week.

Computed in five steps. The denominator of r_{cj} and the inner sum of ℓ are the same number, so it is computed once. With 50 genes every $\mathcal{N}$ can round to zero, so all steps are done in logs (a product becomes a sum, a division becomes a difference):

1. Prior of the cell's week: $p_{t_c} = \pi A^{t_c}$.
2. Log of the numerator of r_{cj}: $a_{cj} = \log p_{t_c}[j] + \log \mathcal{N}(\mathbf{x}_c; \mu_j, U_j)$.
3. Log of the denominator (log-sum-exp): $\ell_c = \log \sum_k e^{a_{ck}} = m_c + \log \sum_k e^{a_{ck} - m_c}$, $m_c = \max_k a_{ck}$.
4. Numerator divided by denominator: $r_{cj} = e^{a_{cj} - \ell_c}$.
5. Log-likelihood: $\ell = \sum_c \ell_c$.

Subtracting the largest term m_c in step 3 makes the largest exponential exactly $e^0 = 1$, so the sum cannot round to zero. Each

step is one line of code (`population.py`); the comments carry the step numbers:

```
# Step 1. Prior of each cell = proportions of its own week: (n, N)
proportions = get_proportions(pi, A, NUMBER_OF_WEEKS)   # (W, N), row t is p_t
priors = proportions[weeks]                              # row c is p_{t_c}
# Step 2. a_cj = log p_{t_c}[j] + log N(x_c; mu_j, U_j): (n, N)
log_densities = log_gaussian_diagonal(X, means, variances)
log_weights = np.log(priors) + log_densities
# Step 3. l_c = log sum_k exp(a_ck): (n,)
row_logsumexp = logsumexp(log_weights, axis=1)
# Step 4. r_cj = exp(a_cj - l_c): (n, N)
r = np.exp(log_weights - row_logsumexp[:, np.newaxis])
# Step 5. l = sum_c l_c
log_likelihood = np.sum(row_logsumexp)
```

`proportions[weeks]` picks row t of the proportions once for every cell measured in week t, so each cell gets the prior of its own week. Worked example, hand-check cell $\mathbf{x} = (3,2)$ from week 1 (toy model of Section 7):

Step	Quantity	MA	T	MU
1	$p_1[j]$	0.7260	0.1665	0.1075
	$\log \mathcal{N}(\mathbf{x}; \mu_j, I)$ (Section 2)	-2.8379	-2.0879	-5.8379
2	$a_j = \log p_1[j] + \log \mathcal{N}$	-3.1581	-3.8806	-8.0681
3	$\ell_c = \log(e^{-3.1581} + e^{-3.8806} + e^{-8.0681})$		-2.7574	
4	$r_j = e^{a_j - \ell_c}$	0.6698	0.3252	0.0049
	check without logs: $p_1[j]\,\mathcal{N}$	0.0425	0.0206	0.0003

Without logs, the three numerators add up to 0.0635; dividing each by this sum gives the same r, and $\log 0.0635 = -2.7574 = \ell_c$. The code reproduces these values and the total $\ell = -12.0361$ of the four hand-check cells.

In the project

Each cell gets three state probabilities. The existing labels (`cell_type`, `cluster`) are used only as an outside check of these probabilities, never as training input.

4 M-step

What I learned

Gaussians (closed form): For every state j and gene g, summing over all cells of all days:

$$\mu_{j,g} = \frac{\sum_c r_{cj} x_{c,g}}{\sum_c r_{cj}}, \qquad \sigma^2_{j,g} = \frac{\sum_c r_{cj} (x_{c,g} - \mu_{j,g})^2}{\sum_c r_{cj}}.$$

This is the update of a plain Gaussian mixture (Bilmes). All days are pooled because the Gaussians are shared. A small lower limit on $\sigma^2_{j,g}$ keeps a variance from shrinking to zero when a state has few cells.

π and A (numerical): With the soft counts $n_t[j] = \sum_{c:t_c=t} r_{cj}$, the M-step maximises

$$Q(\pi, A) = \sum_{t=0}^{3} \sum_j n_t[j] \log p_t[j], \qquad p_t = \pi A^t,$$

with π and every row of A non-negative and summing to 1. If the weights of each week were free, the best p_t would be the soft proportions $n_t[j]/N_t$, where N_t is the number of cells measured in week t. Here they must all come from one π and one A, and A appears as A^2 and A^3. So there is no closed form, and a small numerical optimisation is needed inside the M-step. Baum–Welch has a closed-form $\bar{A}$ only because it counts consecutive pairs of states (ξ_t). Here no cell is seen twice, so there are no pairs. I will write π and each row of A as a softmax of free numbers; the softmax turns any real numbers into positive numbers that sum to 1, so any optimiser keeps them valid. The optimiser does not need the exact maximum: any step that raises Q keeps ℓ from decreasing (generalised EM).

In the project

One EM iteration is: E-step, Gaussian update, then a few optimiser steps on π and A. The log-likelihood check from the toy Baum–Welch carries over: ℓ must never decrease.

5 Starting values

What I learned

When I asked how to initialise π and A, the supervisor suggested $\pi_{\text{start}} = p_0$ and a Householder reflection for A:

$$A_{\text{start}} = I - 2\frac{w w^\top}{w^\top w}, \qquad w = p_0 - p_1,$$

where p_0 and p_1 are rough state proportions on day 0 and day 7. The idea is to start from an A that already turns day 0 into day 7. The matrix has three properties:

- Rows: p_0 and p_1 both sum to 1, so w sums to 0, and every row of A_{start} sums to 1.
- Exactness: it maps p_0 exactly onto p_1 only when the two vectors have the same Euclidean length, which is usually not the case.
- Signs: it can have negative entries. I will replace each negative entry by 0.01 and divide the row by its sum. No entry starts at exactly 0, because a zero blocks that transition.

When π is close to $(1,0,0)$, the start is rough and EM has to do most of the work.

In the project

I will run the Householder start together with several random starts and keep the best ℓ. This is also the anchoring check from the project plan: the previous student set π, A and B by hand to the expected answer and got that answer back. If the Householder start and the random starts all end at the same A, the result is not an echo of the start.

6 What the data can determine, and how to validate

What I learned

Once the proportions are known, $p_{t+1} = p_t \cdot A$ is linear in A. For each column of A, the three steps (day $0 \to 7$, $7 \to 14$, $14 \to 21$) give three equations for three unknowns. So all four time points are needed, including healthy tissue as day 0. Without day 0 there are only two equations per column, and many different A fit the data equally well. Even with four time points, when π is close to $(1,0,0)$, small changes in the proportions cause large changes in the T and MU rows of A. The reason: these rows of A only act on the small part of the population that is already in T or MU. I expect the MA row to be recovered well and the T and MU rows less well. The recovery and day-0 checks in Section 7 test this.

In the project

The validation has three parts:

1. Proportions: modelled p_t against the observed soft proportions, for each of the four days.
2. Gene-by-gene comparison: simulate cells from the fitted model and compare, gene by gene, the simulated and the real distribution of values. Negative simulated values are set to zero.
3. Held-out likelihood: cells kept aside from the start and used only once, at the end.

I will also report how stable each row of A is across restarts and resampled cells. The 50 genes are the top markers of clusters 2 and 8, which also define the labels, so agreement with the labels is partly built in.

7 Toy implementation plan

Toy model

Before the real data, the population model is built and tested on simulated cells. Every cell is a vector of genes from the start, as in the real data. Three stages: 2D (to be checked by hand and in code), then 5D (in code, with one gene that carries no signal), then $d = 50$ on the real data with the same code. The 2D toy model: $\pi = (0.90, 0.05, 0.05)$, gene 1 a myeloid marker and gene 2 a mural marker, diagonal variances of 1, and

$$A = \begin{bmatrix} 0.80 & 0.15 & 0.05 \\ 0.10 & 0.60 & 0.30 \\ 0.02 & 0.03 & 0.95 \end{bmatrix}, \qquad \mu_{\mathrm{MA}} = (4,1), \quad \mu_{\mathrm{T}} = (2.5, 2.5), \quad \mu_{\mathrm{MU}} = (1,4).$$

Cells are simulated one at a time: the state is drawn from p_t of the cell's week, then $\mathbf{x}$ from that state's Gaussian. There is no chain per cell; only the proportions move from week to week. Four hand-check cells, one per week, give reference values for the code.

Checks

The checks, in order:

- Hand values: the code reproduces ℓ, r and the Gaussian update of the hand-check cells.
- Monotone EM: ℓ never decreases.
- Exact proportions: given the exact $p_0, \ldots, p_3$, the π, A step returns the true π and A.
- Recovery: with 300 and then 3000 cells per week, compare the fitted π, A, μ and σ^2 with the truth after matching the states.
- Day 0: drop the day-0 cells and refit from several starts, to see that A is not determined without day 0 (Section 6).
- Test set: compare the held-out ℓ of the best fit with that of the true parameters.

8 Plan for the next two weeks

1. Implement the 2D toy population model (Section 7) and pass the hand-check values.
2. Run the recovery and day-0 experiments on the 2D toy, then extend to 5D.
3. Open `adata_final.h5ad`, select the 50 genes, check the normalisation, and count the cells per state and day.